



Unit 1 - 123

Theory of computation (Anna University)



Scan to open on Studocu



Nutan College Of Engineering & Research,
Talegaon Dabhade, Pune- 410507

DEPARTMENT OF
COMPUTER
SCIENCE&ENGINEERING

Theory of Computations

Unit I – Finite Automata and Regular Expressions

• Introduction

Theory of automata is a theoretical branch of computer science and mathematical. It is the study of abstract machines and the computation problems that can be solved using these machines. The abstract machine is called the automata. An automaton with a finite number of states is called a Finite automaton.

In this subject, we are going to learn how to construct deterministic finite automata, non-deterministic finite automata, Regular expression, context-free grammar, context-free language, Push down automata, Turning machines, etc.

Before learning Automata, you should have a basic understanding of string, language, alphabets, symbols.

Theory of automata is a theoretical branch of computer science and mathematical. It is the study of abstract machines and the computation problems that can be solved using these machines. The abstract machine is called the automata. The main motivation behind developing the automata theory was to develop methods to describe and analyse the dynamic behaviour of discrete systems.

This automaton consists of states and transitions. The **State** is represented by **circles**, and the **Transitions** is represented by **arrows**.

Automata is the kind of machine which takes some string as input and this input goes through a finite number of states and may enter in the final state.

There are the basic terminologies that are important and frequently used in automata:

Symbols:

Symbols are an entity or individual objects, which can be any letter, alphabet or any picture.

Example:

1, a, b, #

Alphabets:

Alphabets are a finite set of symbols. It is denoted by Σ .

Examples:

1. $\Sigma = \{a, b\}$
2. $\Sigma = \{A, B, C, D\}$
3. $\Sigma = \{0, 1, 2\}$
4. $\Sigma = \{0, 1, \dots, 5\}$
5. $\Sigma = \{\#, \beta, \Delta\}$

String:

It is a finite collection of symbols from the alphabet. The string is denoted by w .

Example 1:

If $\Sigma = \{a, b\}$, various string that can be generated from Σ are $\{ab, aa, aaa, bb, bbb, ba, aba, \dots\}$.

- A string with zero occurrences of symbols is known as an empty string. It is represented by ϵ .
- The number of symbols in a string w is called the length of a string. It is denoted by $|w|$.

Example 2:

1. $w = 010$
2. Number of Sting $|w| = 3$

Language:

A language is a collection of appropriate string. A language which is formed over Σ can be **Finite** or **Infinite**.

Example: 1

$L1 = \{\text{Set of string of length 2}\}$

= {aa, bb, ba, bb} **Finite Language**

Example: 2

L2 = {Set of all strings starts with 'a'}

= {a, aa, aaa, abb, abbb, ababb} **Infinite Language**

- **Finite Automata**

- Finite automata are used to recognize patterns.
- It takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs.
- At the time of transition, the automata can either move to the next state or stay in the same state.
- Finite automata have two states, **Accept state** or **Reject state**. When the input string is processed successfully, and the automata reached its final state, then it will accept.

Formal Definition of FA

A finite automaton is a collection of 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

1. Q : finite set of states
2. Σ : finite set of the input symbol
3. q_0 : initial state
4. F : **final** state
5. δ : Transition function

Finite Automata Model:

Finite automata can be represented by input tape and finite control.

Input tape: It is a linear tape having some number of cells. Each input symbol is placed in each cell.

Finite control: The finite control decides the next state on receiving particular input from input tape. The tape reader reads the cells one by one from left to right, and at a time only one input symbol is read.

Types of Automata:

There are two types of finite automata:

1. DFA(deterministic finite automata)
2. NFA(non-deterministic finite automata)

1. DFA

DFA refers to deterministic finite automata. Deterministic refers to the uniqueness of the computation. In the DFA, the machine goes to one state only for a particular input character. DFA does not accept the null move.

2. NFA

NFA stands for non-deterministic finite automata. It is used to transmit any number of states for a particular input. It can accept the null move.

Some important points about DFA and NFA:

1. Every DFA is NFA, but NFA is not DFA.
2. There can be multiple final states in both NFA and DFA.
3. DFA is used in Lexical Analysis in Compiler.
4. NFA is more of a theoretical concept.

Transition Diagram

A transition diagram or state transition diagram is a directed graph which can be constructed as follows:

- There is a node for each state in Q , which is represented by the circle.
- There is a directed edge from node q to node p labeled a if $\delta(q, a) = p$.
- In the start state, there is an arrow with no source.
- Accepting states or final states are indicating by a double circle.

Some Notations that are used in the transition diagram:

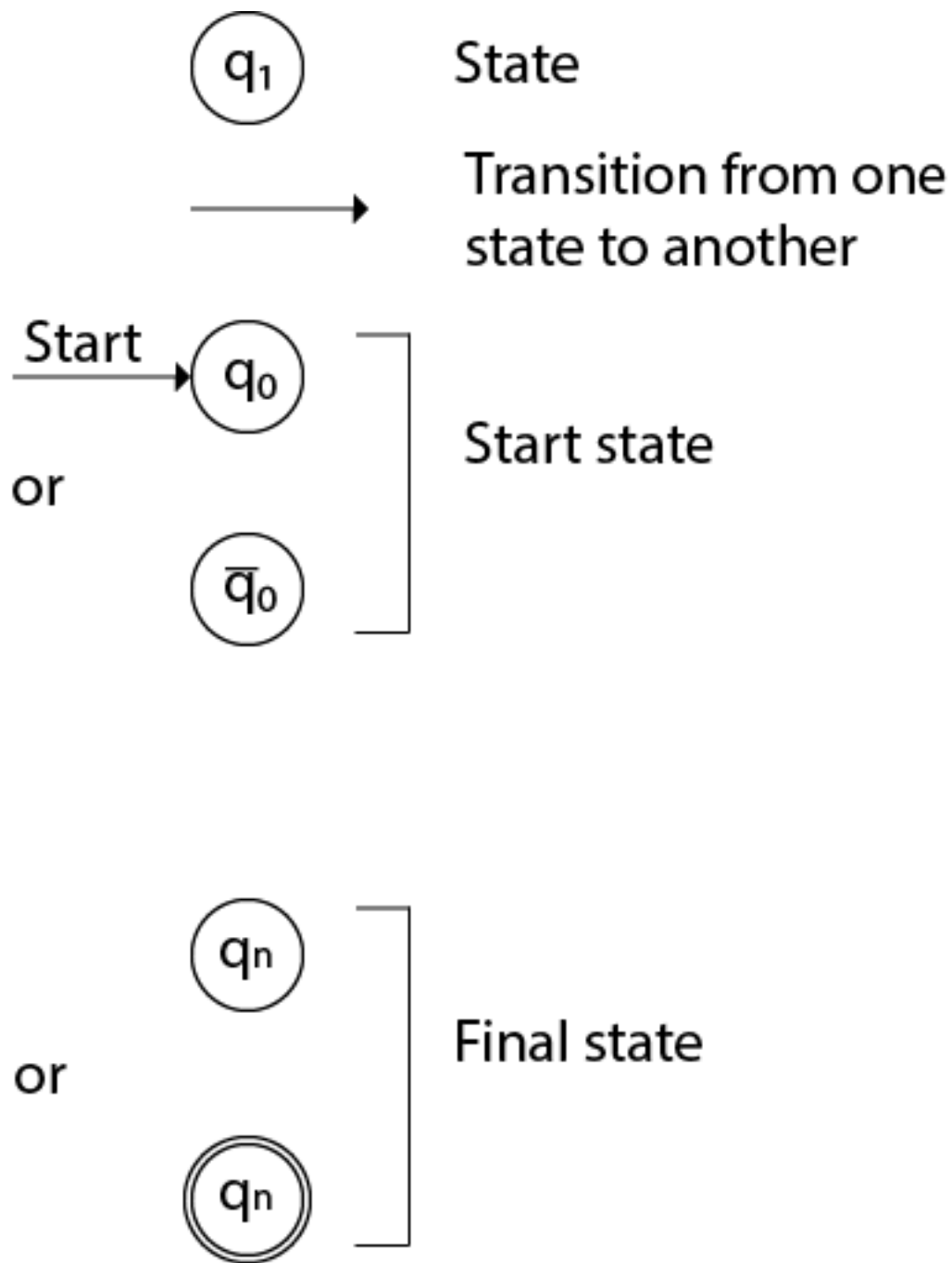


Fig:- Notations

1. In DFA, the input to the automata can be any string. Now, put a pointer to the start state q and read the input string w from left to right and move the pointer according to the transition function, δ . We can read one symbol at a time. If the next symbol of string w is a and the pointer is on state p , move the pointer to $\delta(p, a)$. When the end of the input string w is encountered, then the pointer is on some state F .

2. The string w is said to be accepted by the DFA if $r \in F$ that means the input string w is processed successfully and the automata reached its final state. The string is said to be rejected by DFA if $r \notin F$.

Example 1:

DFA with $\Sigma = \{0, 1\}$ accepts all strings starting with 1.

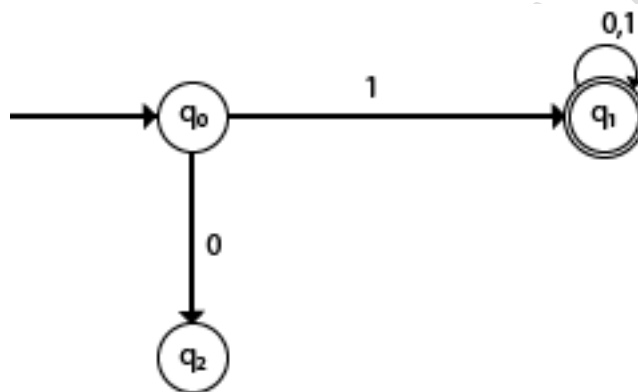


Fig: Transition diagram

The finite automata can be represented using a transition graph. In the above diagram, the machine initially is in start state q_0 then on receiving input 1 the machine changes its state to q_1 . From q_0 on receiving 0, the machine changes its state to q_2 , which is the dead state. From q_1 on receiving input 0, 1 the machine changes its state to q_1 , which is the final state. The possible input strings that can be generated are 10, 11, 110, 101, 111....., that means all string starts with 1.

Transition Table

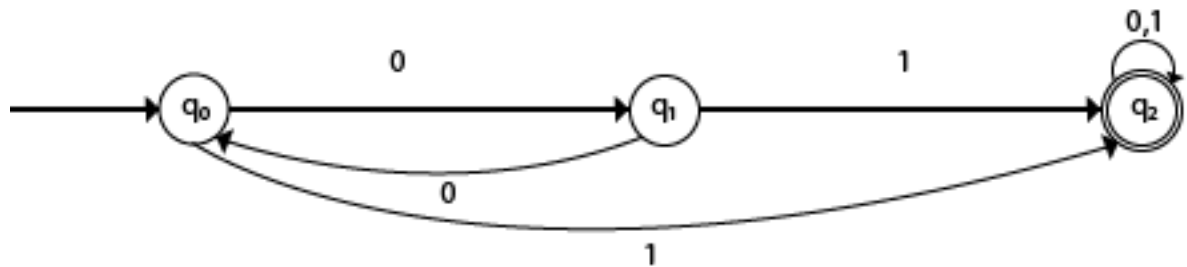
The transition table is basically a tabular representation of the transition function. It takes two arguments (a state and a symbol) and returns a state (the "next state").

A transition table is represented by the following things:

- Columns correspond to input symbols.
- Rows correspond to states.
- Entries correspond to the next state.
- The start state is denoted by an arrow with no source.

- The accept state is denoted by a star.

Example 1



Transition table of given DFA is as follows:

Present State	Next state for Input 0	Next State of Input 1
→q0	q1	q2
q1	q0	q2
*q2	q2	q2

Explanation:

- In the above table, the first column indicates all the current states. Under column 0 and 1, the next states are shown.
- The first row of the transition table can be read as, when the current state is q0, on input 0 the next state will be q1 and on input 1 the next state will be q2.
- In the second row, when the current state is q1, on input 0, the next state will be q0, and on 1 input the next state will be q2.
- In the third row, when the current state is q2 on input 0, the next state will be q2, and on 1 input the next state will be q2.
- The arrow marked to q0 indicates that it is a start state and circle marked to q2 indicates that it is a final state.

- **DFA (Deterministic finite automata)**

- DFA refers to deterministic finite automata. Deterministic refers to the uniqueness of the computation. The finite automata are called deterministic finite automata if the machine is read an input string one symbol at a time.
- In DFA, there is only one path for specific input from the current state to the next state.
- DFA does not accept the null move, i.e., the DFA cannot change state without any input character.
- DFA can contain multiple final states. It is used in Lexical Analysis in Compiler.

In the following diagram, we can see that from state q_0 for input a , there is only one path which is going to q_1 . Similarly, from q_0 , there is only one path for input b going to q_2 .

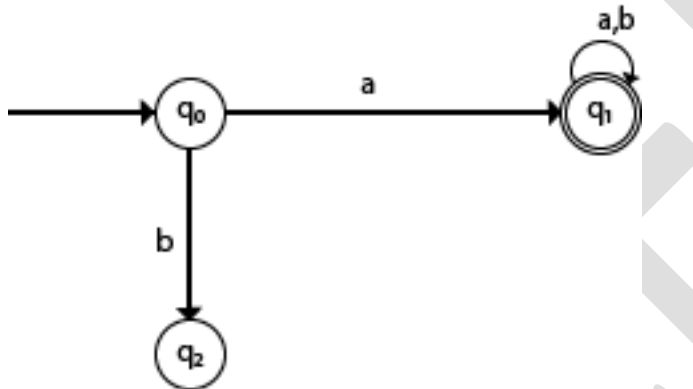


Fig:- DFA

Formal Definition of DFA

A DFA is a collection of 5-tuples same as we described in the definition of FA.

1. Q : finite set of states
2. Σ : finite set of the input symbol
3. q_0 : initial state
4. F : **final** state
5. δ : Transition function

Transition function can be defined as:

1. $\delta: Q \times \Sigma \rightarrow Q$

Graphical Representation of DFA

A DFA can be represented by digraphs called state diagram. In which:

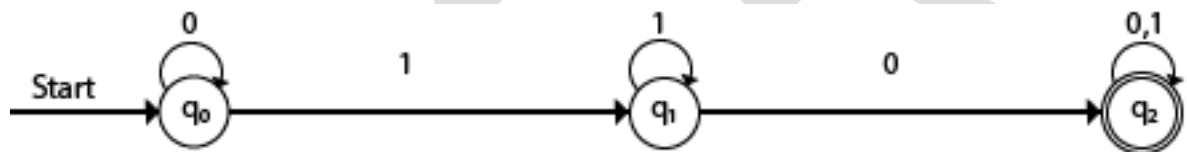
1. The state is represented by vertices.
2. The arc labeled with an input character show the transitions.
3. The initial state is marked with an arrow.
4. The final state is denoted by a double circle.

Example 1:

1. $Q = \{q_0, q_1, q_2\}$
2. $\Sigma = \{0, 1\}$
3. $q_0 = \{q_0\}$
4. $F = \{q_2\}$

Solution:

Transition Diagram:



Transition Table:

Present State	Next state for Input 0	Next State of Input 1
$\rightarrow q_0$	q_0	q_1
q_1	q_2	q_1
$*q_2$	q_2	q_2

Examples of DFA

Example 1:

Design a FA with $\Sigma = \{0, 1\}$ accepts those string which starts with 1 and ends with 0.

Solution:

The FA will have a start state q_0 from which only the edge with input 1 will go to the next state.

- **NFA (Non-Deterministic finite automata)**

- NFA stands for non-deterministic finite automata. It is easy to construct an NFA than DFA for a given regular language.
- The finite automata are called NFA when there exist many paths for specific input from the current state to the next state.
- Every NFA is not DFA, but each NFA can be translated into DFA.
- NFA is defined in the same way as DFA but with the following two exceptions, it contains multiple next states, and it contains ϵ transition.

In the following image, we can see that from state q_0 for input a , there are two next states q_1 and q_2 , similarly, from q_0 for input b , the next states are q_0 and q_1 . Thus it is not fixed or determined that with a particular input where to go next. Hence this FA is called non-deterministic finite automata.

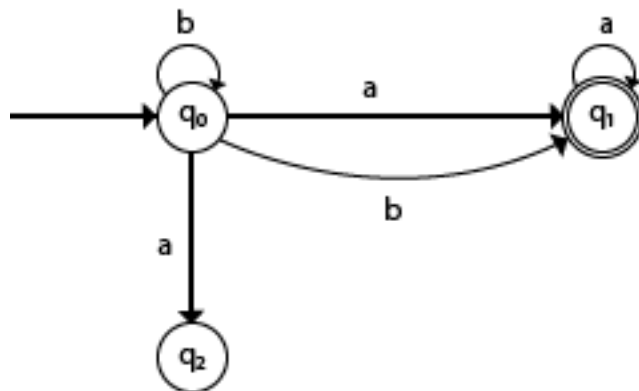


Fig:- NDFA

Formal definition of NFA:

NFA also has five states same as DFA, but with different transition function, as shown follows:

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

where,

1. Q : finite set of states
2. Σ : finite set of the input symbol
3. q_0 : initial state
4. F : **final** state
5. δ : Transition function

Graphical Representation of an NFA

An NFA can be represented by digraphs called state diagram. In which:

1. The state is represented by vertices.
2. The arc labeled with an input character show the transitions.
3. The initial state is marked with an arrow.
4. The final state is denoted by the double circle.

Example 1:

1. $Q = \{q_0, q_1, q_2\}$
2. $\Sigma = \{0, 1\}$
3. $q_0 = \{q_0\}$
4. $F = \{q_2\}$

Solution:

Transition diagram:

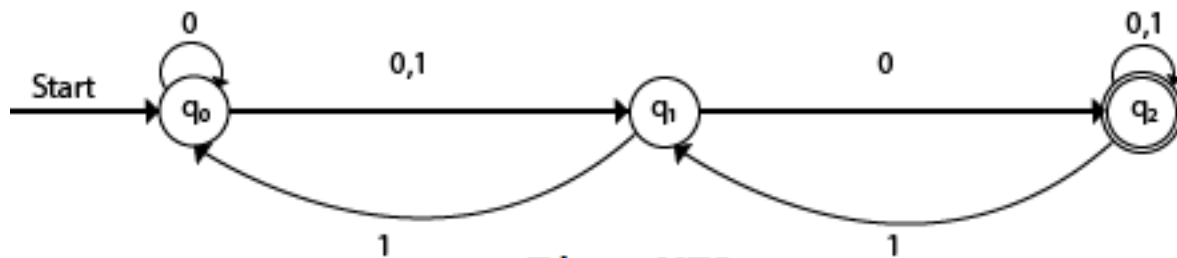


Fig: NFA

Transition Table:

Present State	Next state for Input 0	Next State of Input 1
$\rightarrow q_0$	q_0, q_1	q_1
q_1	q_2	q_0
$*q_2$	q_2	q_1, q_2

In the above diagram, we can see that when the current state is q_0 , on input 0, the next state will be q_0 or q_1 , and on 1 input the next state will be q_1 . When the current state is q_1 , on input 0 the

next state will be q_2 and on 1 input, the next state will be q_0 . When the current state is q_2 , on 0 input the next state is q_2 , and on 1 input the next state will be q_1 or q_2 .

Examples of NFA

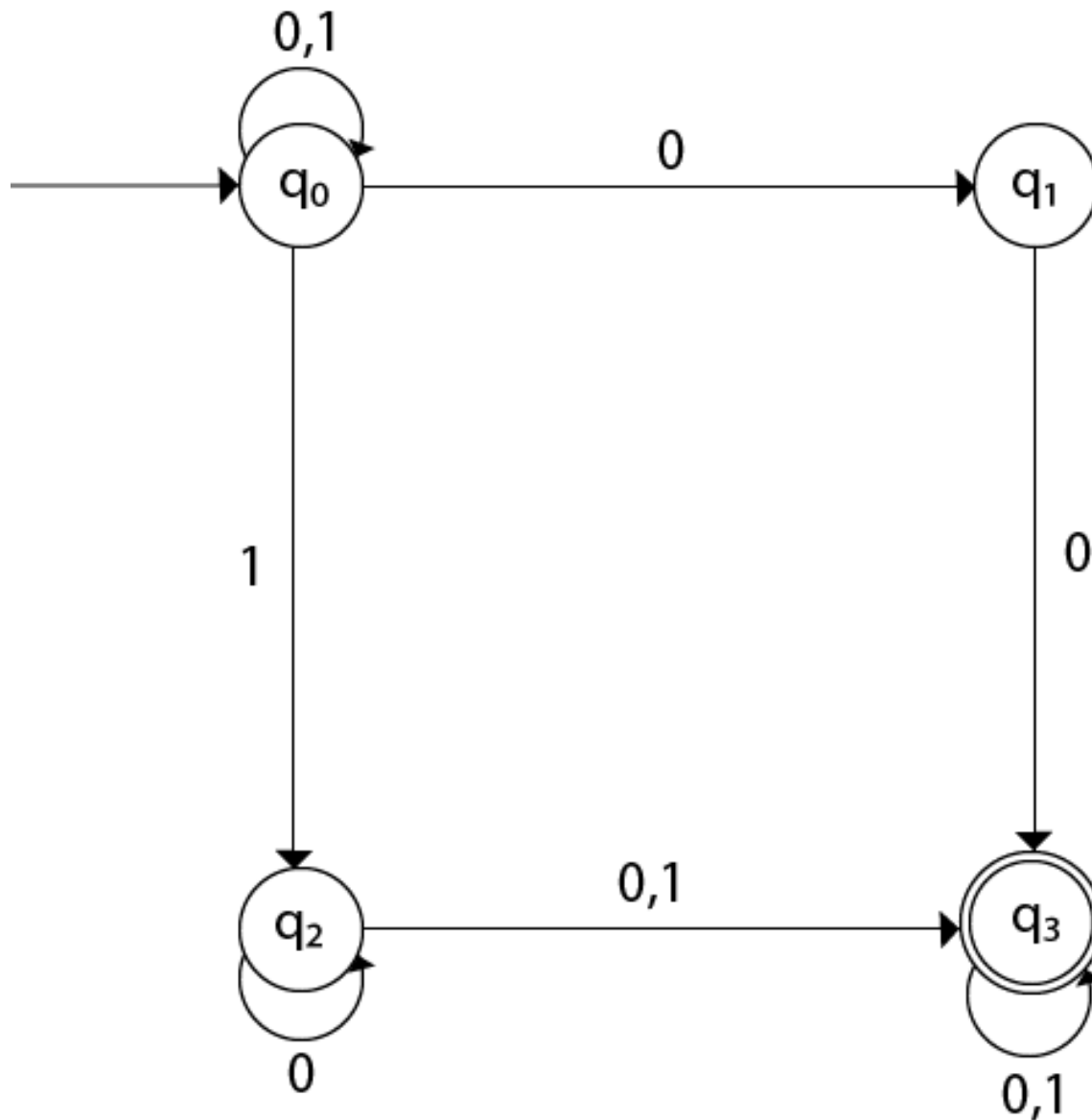
Example 1:

Design a NFA for the transition table as given below:

Present State	0	1
$\rightarrow q_0$	q_0, q_1	q_0, q_2
q_1	q_3	E
q_2	q_2, q_3	q_3
$\rightarrow q_3$	q_3	q_3

Solution:

The transition diagram can be drawn by using the mapping function as given in the table.



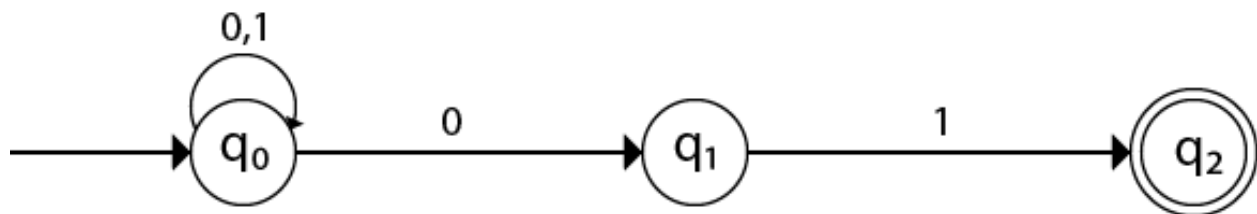
Here,

1. $\delta(q_0, 0) = \{q_0, q_1\}$
2. $\delta(q_0, 1) = \{q_0, q_2\}$
3. Then, $\delta(q_1, 0) = \{q_3\}$
4. Then, $\delta(q_2, 0) = \{q_2, q_3\}$
5. $\delta(q_2, 1) = \{q_3\}$
6. Then, $\delta(q_3, 0) = \{q_3\}$
7. $\delta(q_3, 1) = \{q_3\}$

Example 2:

Design an NFA with $\Sigma = \{0, 1\}$ accepts all string ending with 01.

Solution:



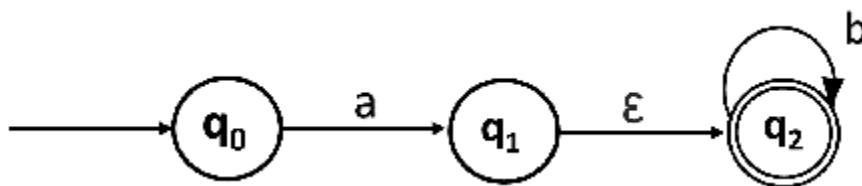
• Eliminating ϵ Transitions

NFA with ϵ can be converted to NFA without ϵ , and this NFA without ϵ can be converted to DFA. To do this, we will use a method, which can remove all the ϵ transition from given NFA. The method will be:

1. Find out all the ϵ transitions from each state from Q . That will be called as ϵ -closure $\{q_i\}$ where $q_i \in Q$.
2. Then δ' transitions can be obtained. The δ' transitions mean a ϵ -closure on δ moves.
3. Repeat Step-2 for each input symbol and each state of given NFA.
4. Using the resultant states, the transition table for equivalent NFA without ϵ can be built.

Example:

Convert the following NFA with ϵ to NFA without ϵ .



Solutions: We will first obtain ϵ -closures of q_0 , q_1 and q_2 as follows:

1. ϵ -closure(q_0) = $\{q_0\}$
2. ϵ -closure(q_1) = $\{q_1, q_2\}$
3. ϵ -closure(q_2) = $\{q_2\}$

Now the δ' transition on each input symbol is obtained as:

1. $\delta'(q_0, a) = \epsilon\text{-closure}(\delta(\delta^*(q_0, \epsilon), a))$
2. $= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_0), a))$
3. $= \epsilon\text{-closure}(\delta(q_0, a))$
4. $= \epsilon\text{-closure}(q_1)$
5. $= \{q_1, q_2\}$
- 6.
7. $\delta'(q_0, b) = \epsilon\text{-closure}(\delta(\delta^*(q_0, \epsilon), b))$
8. $= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_0), b))$
9. $= \epsilon\text{-closure}(\delta(q_0, b))$
10. $= \Phi$

Now the δ' transition on q_1 is obtained as:

1. $\delta'(q_1, a) = \epsilon\text{-closure}(\delta(\delta^*(q_1, \epsilon), a))$
2. $= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_1), a))$
3. $= \epsilon\text{-closure}(\delta(q_1, q_2), a)$
4. $= \epsilon\text{-closure}(\delta(q_1, a) \cup \delta(q_2, a))$
5. $= \epsilon\text{-closure}(\Phi \cup \Phi)$
6. $= \Phi$
- 7.
8. $\delta'(q_1, b) = \epsilon\text{-closure}(\delta(\delta^*(q_1, \epsilon), b))$
9. $= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_1), b))$
10. $= \epsilon\text{-closure}(\delta(q_1, q_2), b)$
11. $= \epsilon\text{-closure}(\delta(q_1, b) \cup \delta(q_2, b))$
12. $= \epsilon\text{-closure}(\Phi \cup q_2)$
13. $= \{q_2\}$

The δ' transition on q_2 is obtained as:

1. $\delta'(q_2, a) = \epsilon\text{-closure}(\delta(\delta^*(q_2, \epsilon), a))$
2. $= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_2), a))$
3. $= \epsilon\text{-closure}(\delta(q_2, a))$
4. $= \epsilon\text{-closure}(\Phi)$
5. $= \Phi$
- 6.
7. $\delta'(q_2, b) = \epsilon\text{-closure}(\delta(\delta^*(q_2, \epsilon), b))$
8. $= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_2), b))$
9. $= \epsilon\text{-closure}(\delta(q_2, b))$
10. $= \epsilon\text{-closure}(q_2)$
11. $= \{q_2\}$

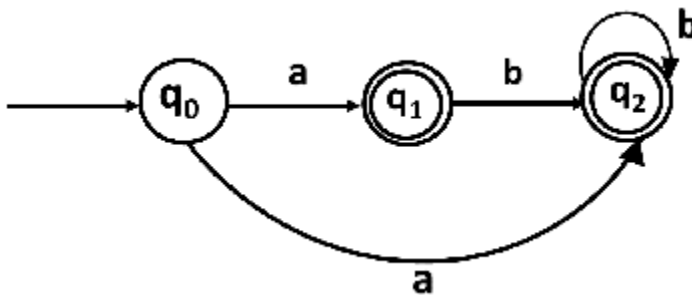
Now we will summarize all the computed δ' transitions:

1. $\delta'(q_0, a) = \{q_0, q_1\}$
2. $\delta'(q_0, b) = \Phi$
3. $\delta'(q_1, a) = \Phi$
4. $\delta'(q_1, b) = \{q_2\}$
5. $\delta'(q_2, a) = \Phi$
6. $\delta'(q_2, b) = \{q_2\}$

The transition table can be:

States	A	b
$\rightarrow q_0$	$\{q_1, q_2\}$	Φ
* q_1	Φ	$\{q_2\}$
* q_2	Φ	$\{q_2\}$

State q_1 and q_2 become the final state as ϵ -closure of q_1 and q_2 contain the final state q_2 . The NFA can be shown by the following transition diagram:



Conversion from NFA to DFA

In this section, we will discuss the method of converting NFA to its equivalent DFA. In NFA, when a specific input is given to the current state, the machine goes to multiple states. It can have zero, one or more than one move on a given input symbol. On the other hand, in DFA, when a specific input is given to the current state, the machine goes to only one state. DFA has only one move on a given input symbol.

Let, $M = (Q, \Sigma, \delta, q_0, F)$ is an NFA which accepts the language $L(M)$. There should be equivalent DFA denoted by $M' = (Q', \Sigma', q_0', \delta', F')$ such that $L(M) = L(M')$.

Steps for converting NFA to DFA:

Step 1: Initially $Q' = \phi$

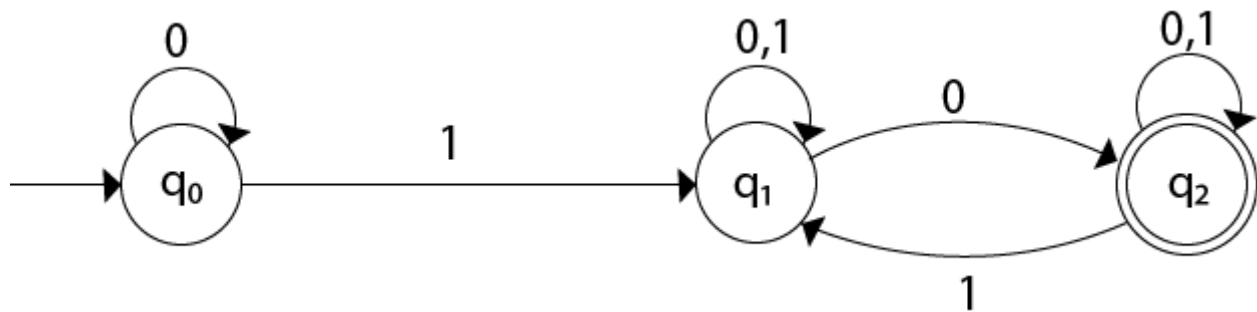
Step 2: Add q_0 of NFA to Q' . Then find the transitions from this start state.

Step 3: In Q' , find the possible set of states for each input symbol. If this set of states is not in Q' , then add it to Q' .

Step 4: In DFA, the final state will be all the states which contain F (final states of NFA)

Example 1:

Convert the given NFA to DFA.



Solution: For the given transition diagram we will first construct the transition table.

State	0	1
$\rightarrow q_0$	q_0	q_1
q_1	$\{q_1, q_2\}$	q_1
$*q_2$	q_2	$\{q_1, q_2\}$

Now we will obtain δ' transition for state q_0 .

1. $\delta'([q_0], 0) = [q_0]$
2. $\delta'([q_0], 1) = [q_1]$

The δ' transition for state q_1 is obtained as:

1. $\delta'([q_1], 0) = [q_1, q_2]$ (new state generated)
2. $\delta'([q_1], 1) = [q_1]$

The δ' transition for state q_2 is obtained as:

1. $\delta'([q_2], 0) = [q_2]$

$$2. \delta'([q2], 1) = [q1, q2]$$

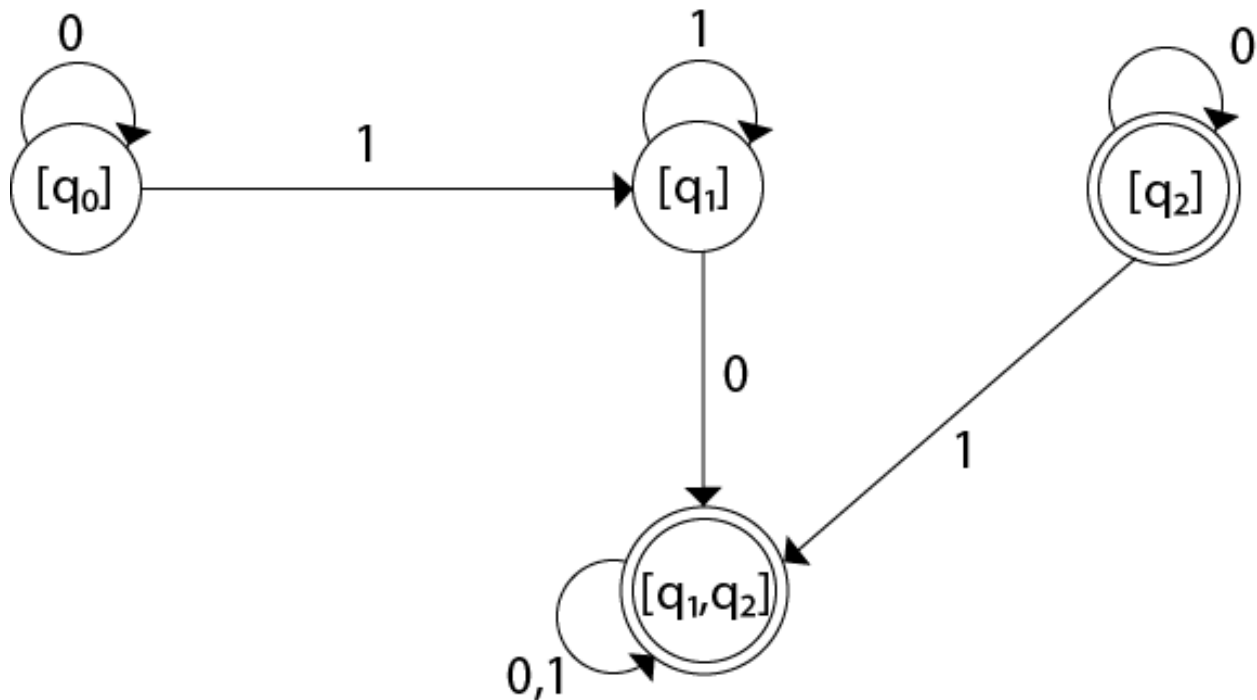
Now we will obtain δ' transition on $[q1, q2]$.

1. $\delta'([q1, q2], 0) = \delta(q1, 0) \cup \delta(q2, 0)$
2. $\quad \quad \quad = \{q1, q2\} \cup \{q2\}$
3. $\quad \quad \quad = [q1, q2]$
4. $\delta'([q1, q2], 1) = \delta(q1, 1) \cup \delta(q2, 1)$
5. $\quad \quad \quad = \{q1\} \cup \{q1, q2\}$
6. $\quad \quad \quad = \{q1, q2\}$
7. $\quad \quad \quad = [q1, q2]$

The state $[q1, q2]$ is the final state as well because it contains a final state $q2$. The transition table for the constructed DFA will be:

State	0	1
$\rightarrow[q0]$	$[q0]$	$[q1]$
$[q1]$	$[q1, q2]$	$[q1]$
$*[q2]$	$[q2]$	$[q1, q2]$
$*[q1, q2]$	$[q1, q2]$	$[q1, q2]$

The Transition diagram will be:



• Conversion from NFA with ϵ to DFA

Non-deterministic finite automata(NFA) is a finite automata where for some cases when a specific input is given to the current state, the machine goes to multiple states or more than 1 states. It can contain ϵ move. It can be represented as $M = \{ Q, \Sigma, \delta, q_0, F \}$.

Where

1. Q : finite set of states
2. Σ : finite set of the input symbol
3. q_0 : initial state
4. F : **final** state
5. δ : Transition function

NFA with ϵ move: If any FA contains ϵ transaction or move, the finite automata is called NFA with ϵ move.

ϵ -closure: ϵ -closure for a given state A means a set of states which can be reached from the state A with only ϵ (null) move including the state A itself.

Steps for converting NFA with ϵ to DFA:

Step 1: We will take the ϵ -closure for the starting state of NFA as a starting state of DFA.

Step 2: Find the states for each input symbol that can be traversed from the present. That means the union of transition value and their closures for each state of NFA present in the current state of DFA.

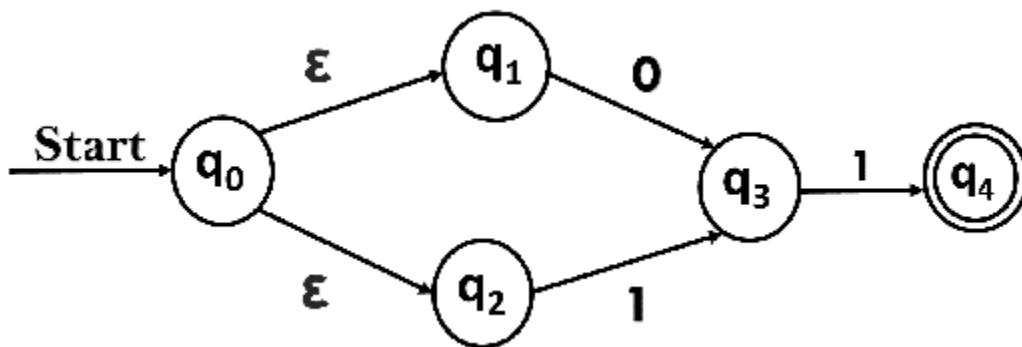
Step 3: If we found a new state, take it as current state and repeat step 2.

Step 4: Repeat Step 2 and Step 3 until there is no new state present in the transition table of DFA.

Step 5: Mark the states of DFA as a final state which contains the final state of NFA.

Example 1:

Convert the NFA with ϵ into its equivalent DFA.



Solution:

Let us obtain ϵ -closure of each state.

1. ϵ -closure $\{q_0\} = \{q_0, q_1, q_2\}$
2. ϵ -closure $\{q_1\} = \{q_1\}$
3. ϵ -closure $\{q_2\} = \{q_2\}$
4. ϵ -closure $\{q_3\} = \{q_3\}$
5. ϵ -closure $\{q_4\} = \{q_4\}$

Now, let ϵ -closure $\{q_0\} = \{q_0, q_1, q_2\}$ be state A.

Hence

$$\begin{aligned}
 \delta'(A, 0) &= \epsilon\text{-closure } \{\delta((q_0, q_1, q_2), 0)\} \\
 &= \epsilon\text{-closure } \{\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0)\} \\
 &= \epsilon\text{-closure } \{q_3\} \\
 &= \{q_3\} \quad \text{call it as state B.}
 \end{aligned}$$

$$\begin{aligned}
 \delta'(A, 1) &= \varepsilon\text{-closure } \{ \delta((q_0, q_1, q_2), 1) \} \\
 &= \varepsilon\text{-closure } \{ \delta((q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1) \} \\
 &= \varepsilon\text{-closure } \{ q_3 \} \\
 &= \{ q_3 \} = B.
 \end{aligned}$$

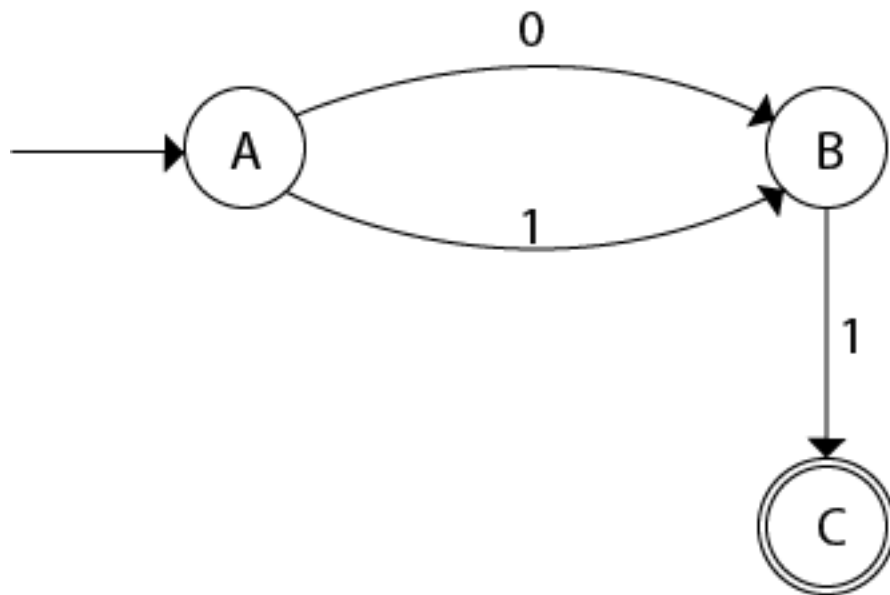
Now,

$$\begin{aligned}
 \delta'(B, 0) &= \varepsilon\text{-closure } \{ \delta(q_3, 0) \} \\
 &= \phi \\
 \delta'(B, 1) &= \varepsilon\text{-closure } \{ \delta(q_3, 1) \} \\
 &= \varepsilon\text{-closure } \{ q_4 \} \\
 &= \{ q_4 \} \quad \textbf{i.e. state C}
 \end{aligned}$$

For state C:

1. $\delta'(C, 0) = \varepsilon\text{-closure } \{ \delta(q_4, 0) \}$
2. $= \phi$
3. $\delta'(C, 1) = \varepsilon\text{-closure } \{ \delta(q_4, 1) \}$
4. $= \phi$

The DFA will be,



• Regular Expression

- The language accepted by finite automata can be easily described by simple expressions called Regular Expressions. It is the most effective way to represent any language.
- The languages accepted by some regular expression are referred to as Regular languages.
- A regular expression can also be described as a sequence of pattern that defines a string.
- Regular expressions are used to match character combinations in strings. String searching algorithm used this pattern to find the operations on a string.

For instance:

In a regular expression, x^* means zero or more occurrence of x . It can generate $\{e, x, xx, xxx, xxxx, \dots\}$

In a regular expression, x^+ means one or more occurrence of x . It can generate $\{x, xx, xxx, xxxx, \dots\}$

Operations on Regular Language

The various operations on regular language are:

Union: If L and M are two regular languages then their union $L \cup M$ is also a union.

1. $L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$

Intersection: If L and M are two regular languages then their intersection is also an intersection.

1. $L \cap M = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$

Kleen closure: If L is a regular language then its Kleen closure L^* will also be a regular language.

1. $L^* = \text{Zero or more occurrence of language } L.$

Example 1:

Write the regular expression for the language accepting all combinations of a 's, over the set $\Sigma = \{a\}$

Solution:

All combinations of a 's means a may be zero, single, double and so on. If a is appearing zero times, that means a null string. That is we expect the set of $\{\epsilon, a, aa, aaa, \dots\}$. So we give a regular expression for this as:

1. $R = a^*$

That is Kleen closure of a.

Example 2:

Write the regular expression for the language accepting all combinations of a's except the null string, over the set $\Sigma = \{a\}$

Solution:

The regular expression has to be built for the language

1. $L = \{a, aa, aaa, \dots\}$

This set indicates that there is no null string. So we can denote regular expression as:

$$R = a^+$$

Examples of Regular Expression

Example 1:

Write the regular expression for the language accepting all the string which are starting with 1 and ending with 0, over $\Sigma = \{0, 1\}$.

Solution:

In a regular expression, the first symbol should be 1, and the last symbol should be 0. The r.e. is as follows:

1. $R = 1(0+1)^*0$

Example 2:

Write the regular expression for the language starting and ending with a and having any having any combination of b's in between.

Solution:

The regular expression will be:

1. $R = a b^* a$

Conversion of RE to FA

To convert the RE to FA, we are going to use a method called the subset method. This method is used to obtain FA from the given regular expression. This method is given below:

Step 1: Design a transition diagram for given regular expression, using NFA with ϵ moves.

Step 2: Convert this NFA with ϵ to NFA without ϵ .

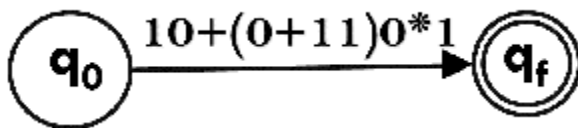
Step 3: Convert the obtained NFA to equivalent DFA.

Example 1:

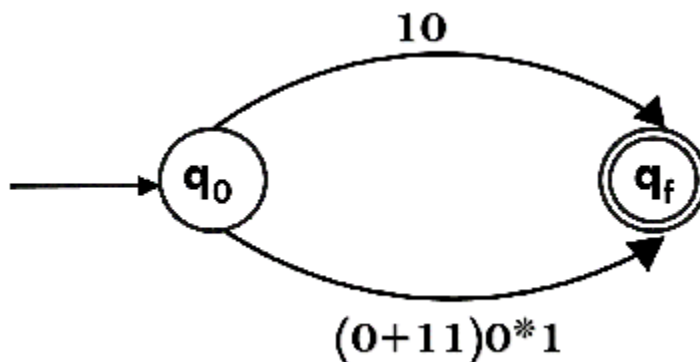
Design a FA from given regular expression $10 + (0 + 11)0^*1$.

Solution: First we will construct the transition diagram for a given regular expression.

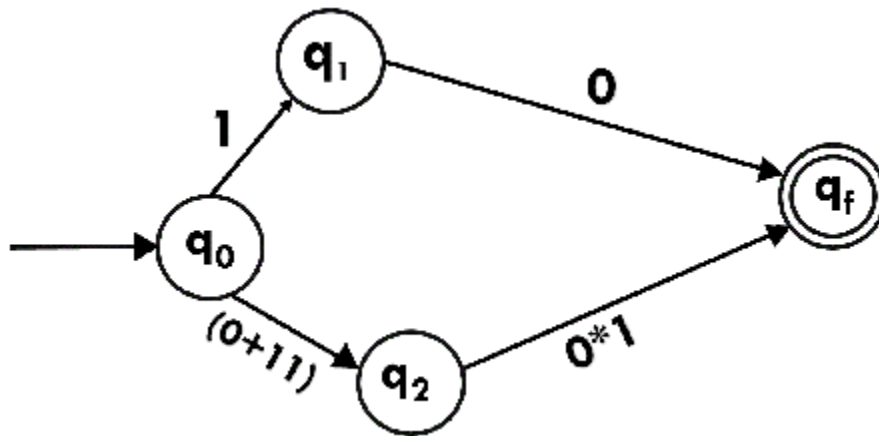
Step 1:



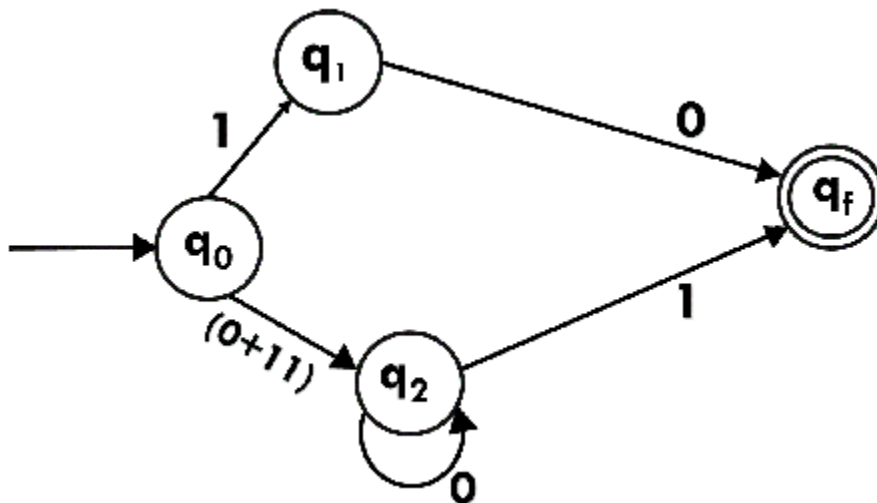
Step 2:



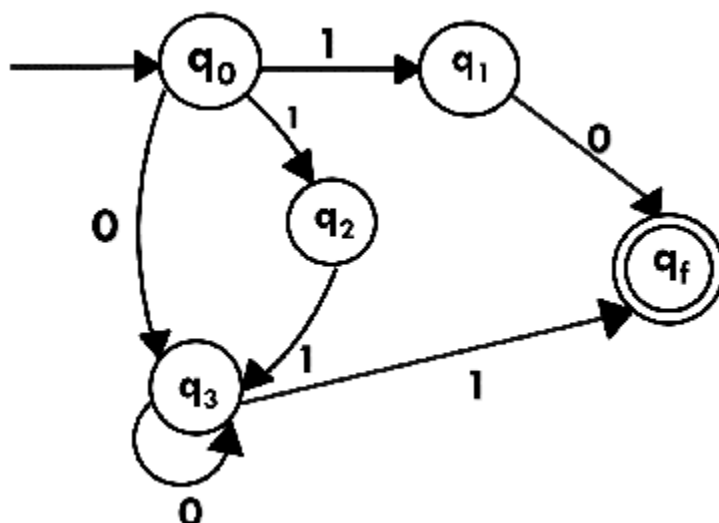
Step 3:



Step 4:



Step 5



Now we have got NFA without ϵ . Now we will convert it into required DFA for that, we will first write a transition table for this NFA.

State	0	1
$\rightarrow q_0$	q_3	$\{q_1, q_2\}$
q_1	Q_f	ϕ
q_2	Φ	q_3
q_3	q_3	q_f
$*q_f$	Φ	ϕ

The equivalent DFA will be:

State	0	1
$\rightarrow [q_0]$	$[q_3]$	$[q_1, q_2]$
$[q_1]$	$[q_f]$	ϕ
$[q_2]$	ϕ	$[q_3]$
$[q_3]$	$[q_3]$	$[q_f]$
$[q_1, q_2]$	$[q_f]$	$[q_f]$
$*[q_f]$	ϕ	ϕ

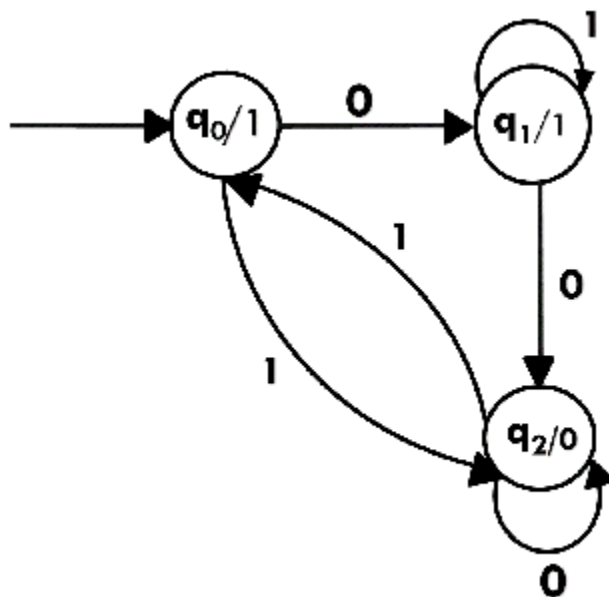
• Moore Machine

Moore machine is a finite state machine in which the next state is decided by the current state and current input symbol. The output symbol at a given time depends only on the present state of the machine. Moore machine can be described by 6 tuples $(Q, q_0, \Sigma, O, \delta, \lambda)$ where,

1. Q : finite set of states
2. q_0 : initial state of machine
3. Σ : finite set of input symbols
4. O : output alphabet
5. δ : transition function where $Q \times \Sigma \rightarrow Q$
6. λ : output function where $Q \rightarrow O$

Example 1:

The state diagram for Moore Machine is



Current State	Next State (δ)		Output(λ)
	0	1	
q_0	q_1	q_2	1
q_1	q_2	q_1	1
q_2	q_2	q_0	0

In the above Moore machine, the output is represented with each input state separated by /. The output length for a Moore machine is greater than input by 1.

Input: 010

Transition: $\delta(q_0, 0) \Rightarrow \delta(q_1, 1) \Rightarrow \delta(q_1, 0) \Rightarrow q_2$

Output: 1110(1 for q_0 , 1 for q_1 , again 1 for q_1 , 0 for q_2)

• Mealy Machine

A Mealy machine is a machine in which output symbol depends upon the present input symbol and present state of the machine. In the Mealy machine, the output is represented with each input symbol for each state separated by /. The Mealy machine can be described by 6 tuples $(Q, q_0, \Sigma, O, \delta, \lambda')$ where

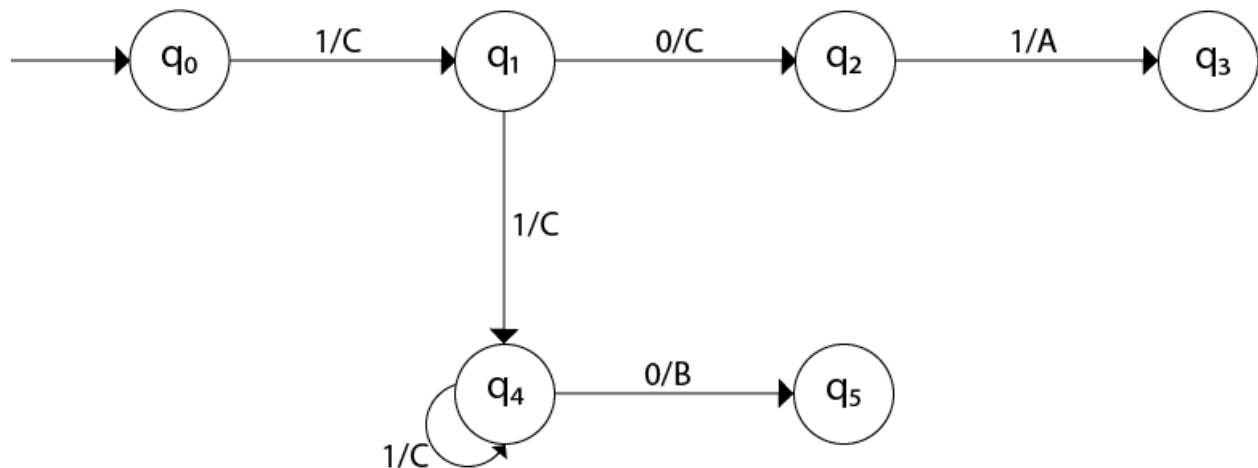
1. Q : finite set of states
2. q_0 : initial state of machine
3. Σ : finite set of input alphabet
4. O : output alphabet
5. δ : transition function where $Q \times \Sigma \rightarrow Q$
6. λ' : output function where $Q \times \Sigma \rightarrow O$

Example 1:

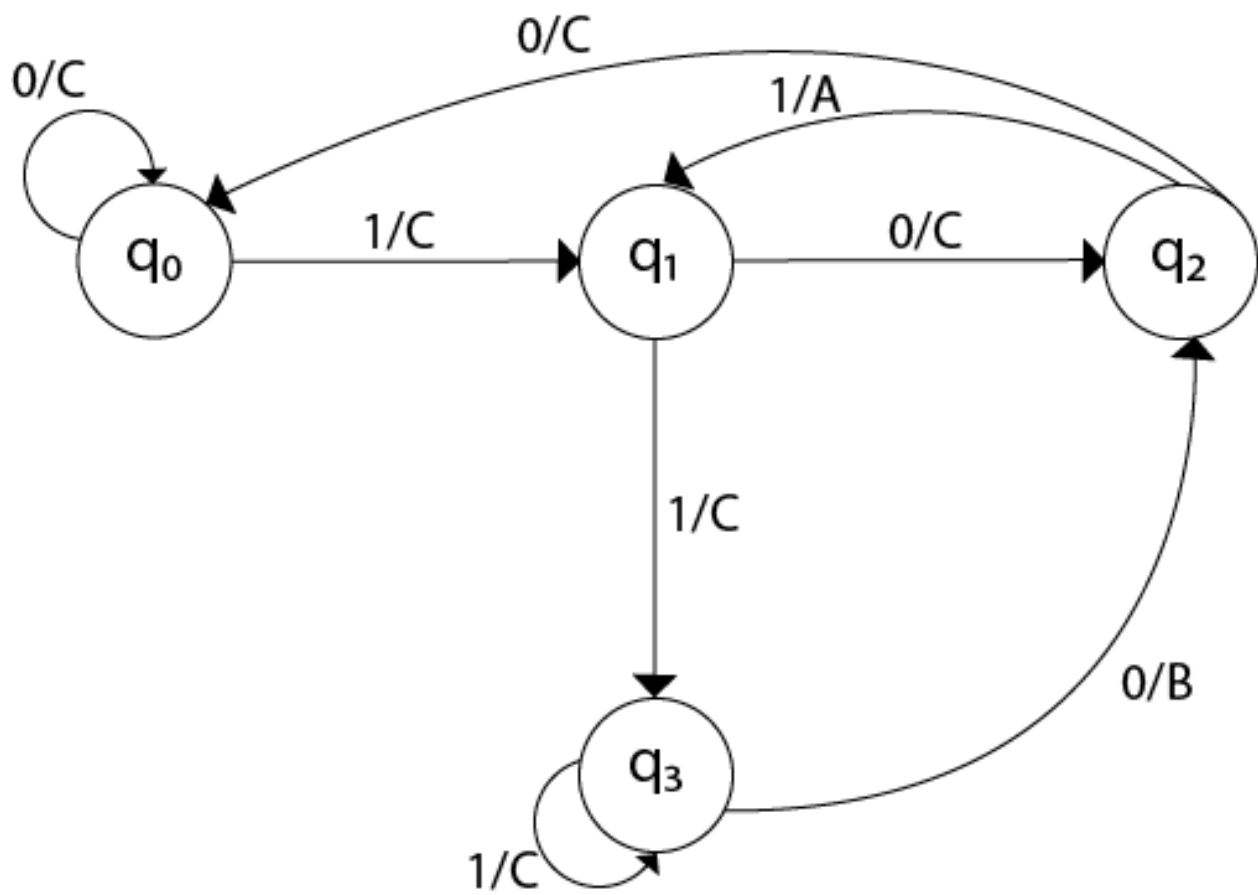
Design a Mealy machine for a binary input sequence such that if it has a substring 101, the machine output A, if the input has substring 110, it outputs B otherwise it outputs C.

Solution: For designing such a machine, we will check two conditions, and those are 101 and 110. If we get 101, the output will be A. If we recognize 110, the output will be B. For other strings the output will be C.

The partial diagram will be:



Now we will insert the possibilities of 0's and 1's for each state. Thus the Mealy machine becomes:



- **Conversion from Mealy machine to Moore Machine**

In Moore machine, the output is associated with every state, and in Mealy machine, the output is given along the edge with input symbol. To convert Moore machine to Mealy machine, state output symbols are distributed to input symbol paths. But while converting the Mealy machine to Moore machine, we will create a separate state for every new output symbol and according to incoming and outgoing edges are distributed.

The following steps are used for converting Mealy machine to the Moore machine:

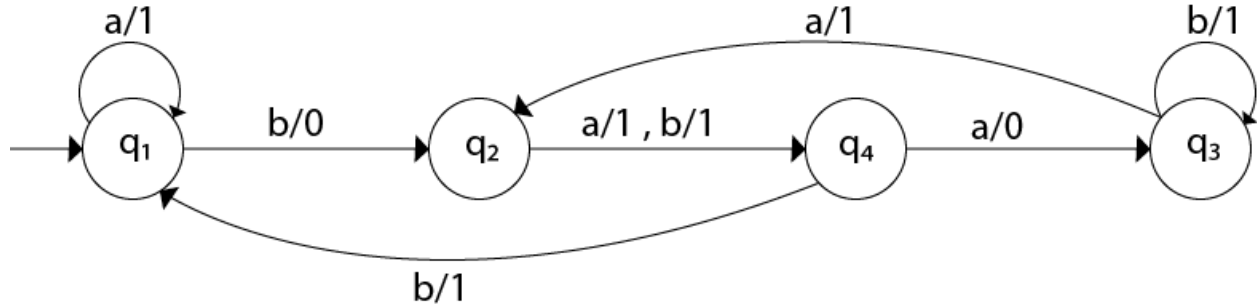
Step 1: For each state(Q_i), calculate the number of different outputs that are available in the transition table of the Mealy machine.

Step 2: Copy state Q_i , if all the outputs of Q_i are the same. Break q_i into n states as Q_{in} , if it has n distinct outputs where $n = 0, 1, 2, \dots$

Step 3: If the output of initial state is 0, insert a new initial state at the starting which gives 1 output.

Example 1:

Convert the following Mealy machine into equivalent Moore machine.



Solution:

Transition table for above Mealy machine is as follows:

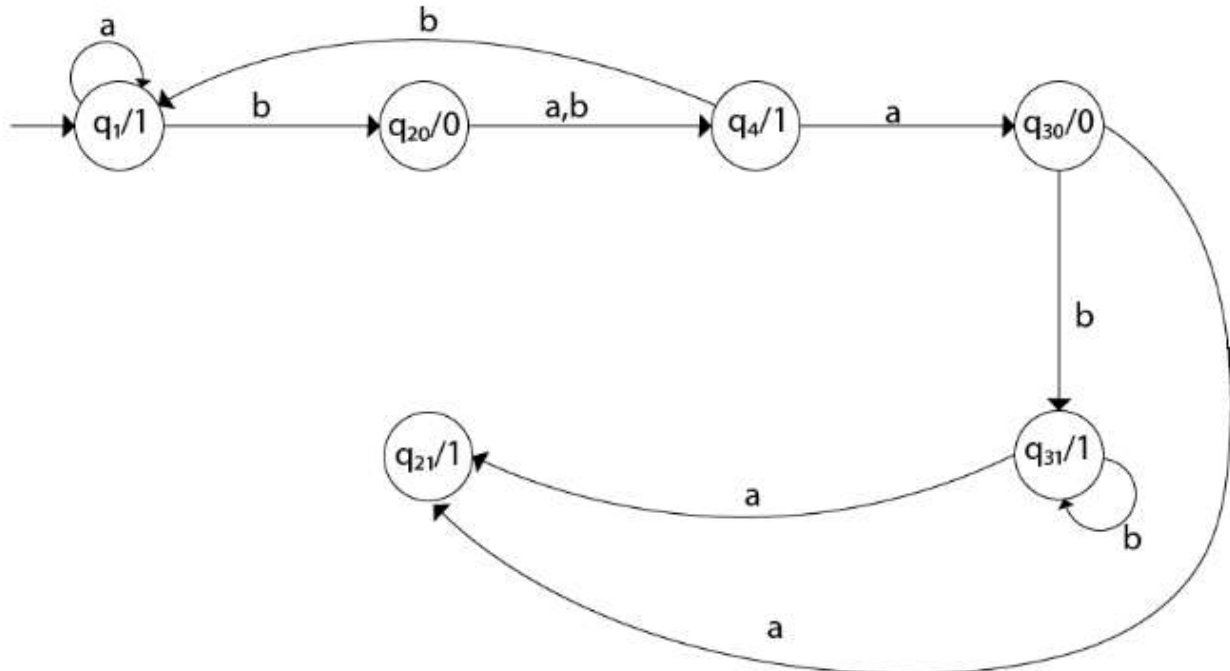
Present State	Next State			
	a		b	
	State	O/P	State	O/P
q ₁	q ₁	1	q ₂	0
q ₂	q ₄	1	q ₄	1
q ₃	q ₂	1	q ₃	1
q ₄	q ₃	0	q ₁	1

- For state q₁, there is only one incident edge with output 0. So, we don't need to split this state in Moore machine.
- For state q₂, there is 2 incident edge with output 0 and 1. So, we will split this state into two states q₂₀(state with output 0) and q₂₁(with output 1).
- For state q₃, there is 2 incident edge with output 0 and 1. So, we will split this state into two states q₃₀(state with output 0) and q₃₁(state with output 1).
- For state q₄, there is only one incident edge with output 0. So, we don't need to split this state in Moore machine.

Transition table for Moore machine will be:

Present State	Next State		Output
	a=0	a=1	
q ₁	q ₁	q ₂	1
q ₂₀	q ₄	q ₄	0
q ₂₁	∅	∅	1
q ₃₀	q ₂₁	q ₃₁	0
q ₃₁	q ₂₁	q ₃₁	1
q ₄	q ₃	q ₄	1

Transition diagram for Moore machine will be:



• Conversion from Moore machine to Mealy Machine

In the Moore machine, the output is associated with every state, and in the mealy machine, the output is given along the edge with input symbol. The equivalence of the Moore machine and Mealy machine means both the machines generate the same output string for same input string.

We cannot directly convert Moore machine to its equivalent Mealy machine because the length of the Moore machine is one longer than the Mealy machine for the given input. To convert Moore machine to Mealy machine, state output symbols are distributed into input symbol paths. We are going to use the following method to convert the Moore machine to Mealy machine.

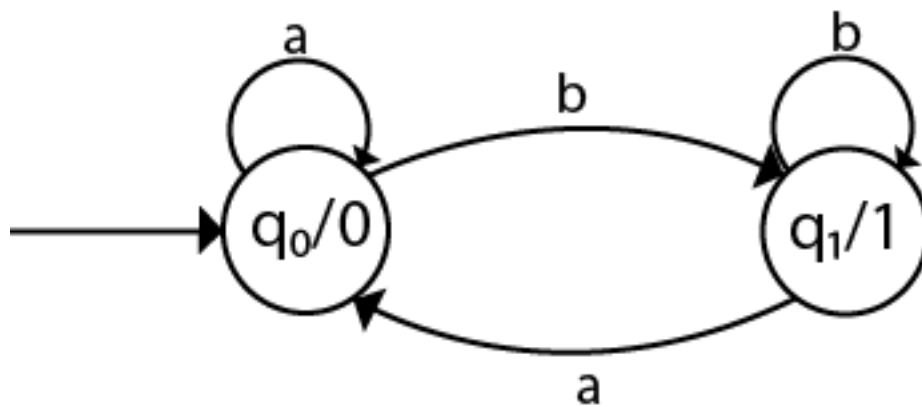
Method for conversion of Moore machine to Mealy machine

Let $M = (Q, \Sigma, \delta, \lambda, q_0)$ be a Moore machine. The equivalent Mealy machine can be represented by $M' = (Q, \Sigma, \delta, \lambda', q_0)$. The output function λ' can be obtained as:

1. $\lambda'(q, a) = \lambda(\delta(q, a))$

Example 1:

Convert the following Moore machine into its equivalent Mealy machine.



Solution:

The transition table of given Moore machine is as follows:

Q	a	B	Output(λ)
q0	q0	q1	0
q1	q0	q1	1

The equivalent Mealy machine can be obtained as follows:

1. $\lambda'(q_0, a) = \lambda(\delta(q_0, a))$
2. $= \lambda(q_0)$
3. $= 0$
- 4.
5. $\lambda'(q_0, b) = \lambda(\delta(q_0, b))$
6. $= \lambda(q_1)$
7. $= 1$

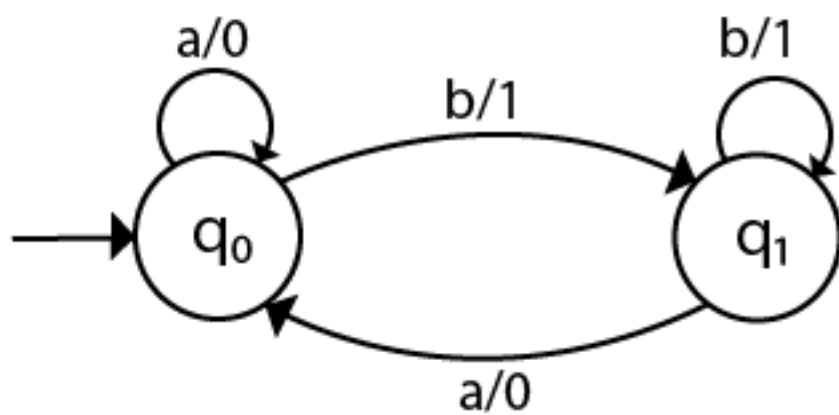
The λ for state q_1 is as follows:

1. $\lambda'(q_1, a) = \lambda(\delta(q_1, a))$
2. $= \lambda(q_0)$
3. $= 0$
- 4.
5. $\lambda'(q_1, b) = \lambda(\delta(q_1, b))$
6. $= \lambda(q_1)$
7. $= 1$

Hence the transition table for the Mealy machine can be drawn as follows:

Q \ Σ	Input 0		Input 1	
	State	O/P	State	O/P
q_0	q_0	0	q_1	1
q_1	q_0	0	q_1	1

The equivalent Mealy machine will be,



Prof.S.D.Shingare
(Subject Incharge)